

OPERATOR'S MANUAL · V02

Claude SEO Pro

The complete, hands-on guide to operating the evidence-verified SEO command center — from a blank machine to delivering white-labeled, client-grade audit reports. Written for an SEO manager with **no coding background**.

Manual version: v02 · **Product version:** 1.2.0 · **Date:** June 2026

Supersedes: Manual v01 (every section rewritten and expanded)

Interactive HTML: use the sidebar to navigate and the filter box to search sections. Collapsible boxes (▶) hide deep-dives until you need them.

Claude SEO Pro turns Claude Code into a complete SEO audit department: 30 skills, 19 specialist agents, live data from DataForSEO and Google, persistent per-client memory, and client-grade PDF reports — run by **one person, from one command line**, with every number in every report traceable to evidence.

The one story that explains every design decision

An early audit shipped findings that turned out to be **fabricated**. Subagents couldn't fetch the website, silently substituted "typical patterns for sites of this type," and the orchestrator stripped their warning caveats — promoting guesses to "Critical Issues." The client caught it.

Everything distinctive about this system is a countermeasure to that incident:

- **Evidence Integrity Protocol** — every finding cites a captured file and line, or a named API response field. No source → it doesn't ship.
- **"Data pending", never guesses** — a blocked API produces an honest pending section with the exact error code and the unlock step.
- **A machine-enforced report contract** — a linter rejects reports that miss sections or compress into summaries.
- **Receipts** — every report's appendix lists every API call it made, with per-call status.

WHAT THIS MEANS FOR YOU

When a client asks "how do you know?", the answer is in the report. That is the product.

What's new since Manual v01

ADDITION (V1.1.0)	WHERE IN THIS MANUAL
NEW Report depth contract + machine linter (14 mandatory sections)	\$8
NEW White-label branding for client reports	\$8.4
NEW Install drift guard (catches stale installs)	\$5.2
NEW CI regression gate (68-assertion suite on every push)	\$15
NEW Plugin-marketplace one-command install	\$5.1
NEW Roadmap + refused-features guardrail	\$15
Fixed: keyword-research preflight bug; timezone deprecations; stale-install incident	\$17
NEW 1.2.0 Claude ↔ OpenRouter backend switching (out-of-credits continuity + custom frontier models)	\$12.5 · \$6.8

2 · How to use this manual

- **New operator?** Read §3 (quick start) → §5 (install) → §6 (onboarding) → §7 (first audit). That's a working system in under an hour.
- **Looking something up?** Type in the sidebar filter box, or jump to §9–10 (commands), §17 (errors).
- **Conventions:** `like this` = type it exactly. `Enter` = key press. Boxes marked ▶ expand on click. **PRO** = added by this distribution; **UPSTREAM** = inherited from the open-source core; **NEW** = added in v1.1.0.
- **Terminals:** commands shown for Windows PowerShell; macOS/Linux differences are noted inline. Inside Claude Code, prefix any shell command with `!` to run it in your session.

THE TWO PROMPTS YOU'LL LIVE IN

Claude Code chat — where you type `/seo` commands and converse. **Your terminal** — where you run installers, the wizard, and Python tools. Secrets are only ever typed in the terminal (masked), never in chat.

3 · The 15-minute quick start

- 1 **Install** (Windows; see §5 for macOS and marketplace paths):

```
git clone https://github.com/creator-imran/claude-seo-pro.git
powershell -ExecutionPolicy Bypass -File claude-seo-pro\install.ps1
```

- 2 **Onboard your APIs.** Start Claude Code (`claude`) and type `/seo-setup` . Claude walks you through each provider — minimum useful set: **DataForSEO** (rankings/keywords) + **Google API key** (real Core Web Vitals). Everything is optional; skip what you don't have.

- 3 **Verify:** `/seo-setup verify` — you want green on the providers you configured, and `[INSTALL FRESH]` from the drift check.

- 4 **Run your first audit:**

```
/seo audit https://yourclient.com
```

Expect 10–20 minutes: pre-fetch → parallel specialists → live data → report + PDF. Full walkthrough in §7.

- 5 **Open the PDF** (path is stated at the end of the run), skim the Executive Summary, then read §8 to understand every section.

IF ANYTHING FAILS

§17 is the troubleshooting encyclopedia. The single most common first-day issue: DataForSEO's **IP whitelist** (`40207`) — see §17.

4 · Core concepts (10 minutes that make everything else obvious)

4.1 Architecture: skills, agents, engines

Three kinds of moving parts:

PART	WHAT IT IS	EXAMPLES
Skills (30)	Instruction packs that teach Claude a workflow. You invoke them as <code>/seo ...</code> commands.	<code>seo-audit</code> , <code>seo-schema</code> , PRO <code>seo-setup</code> , <code>seo-knowledge</code>
Agents (19)	Specialist sub-workers a skill spawns in parallel — each analyzes one dimension (technical, content, schema...) and reports back with evidence.	<code>seo-technical</code> , <code>seo-content</code> , PRO <code>seo-learn</code>
Engines (4)	PRO Python packages that do deterministic work: onboarding/credentials, client knowledge, model routing, the Slack connector.	<code>onboarding/</code> , <code>knowledge/</code> , <code>routing/</code> , <code>connector/</code>

Where things live on your machine after install:

```
~/ .claude/skills/           ← the 30 skills (seo, seo-audit, seo-setup, ...)  
~/ .claude/skills/seo/      ← shared scripts + the 4 Pro engines  
~/ .claude/agents/         ← the 19 agent definitions  
~/ .config/claude-seo/     ← YOUR data: API keys (owner-only), client knowledge,  
                           cache, branding.json, install-manifest.json
```

THE GOLDEN SEPARATION

The **repo** (claude-seo-pro folder) is the product. **~/ .claude** is the installed copy. **~/ .config/claude-seo** is your private data — keys and client knowledge. The repo never contains your data; updates never touch your data.

4.2 The Evidence Integrity Protocol

Eight rules wired into the audit skill itself. Worth knowing them, because they explain "odd" behaviors that are actually the system protecting you:

The 8 rules, and what each one looks like in practice

#	RULE	WHAT YOU'LL OBSERVE
1	Pre-fetch all pages first; specialists read local files only	The audit starts by downloading dozens of pages to disk before any analysis
2	A failed fetch is reported, never substituted	"Could not fetch /pricing (403)" instead of invented findings about the pricing page
3	Every finding cites evidence (file:line or API field)	Each finding card has a small "Evidence:" line
4	Specialist caveats are never stripped	You may see [VERIFY] markers survive into drafts — that's intentional
5	No score without backing data	Backlinks shows PENDING in the scorecard rather than a made-up number
6	CWV uses real-user field data; lab labeled as lab	"Lab LCP 18s (throttling artifact — field is 2.4s)"
7	Schema is read from raw HTML	The fetcher saves raw HTML because rendered fetches strip JSON-LD
8	What was NOT covered is reported	"Crawl sampled 50 of 181 sitemap URLs" in the appendix

4.3 The 4-phase audit pipeline

A full `/seo audit` is not one step — it's four phases, in order:

PHASE	NAME	WHAT HAPPENS	OUTPUT
0	Business Intelligence	Recalls everything already known about this client from the knowledge store, then infers business model, country, target markets, ideal customers, seed keyword themes — each with confidence + evidence	<code>business-profile.json</code>
1	Evidence audit	Pre-fetches all money pages + sitemaps + robots; spawns parallel specialists (cheap models for extraction, stronger for reasoning); adversarially verifies Critical/High findings	per-dimension findings, all cited
2	Keyword research	Live DataForSEO across the origin + detected markets: volumes, CPC, difficulty, intent, competitor gap; tiered into quick-wins / rescues / gaps	keyword tables + opportunity tiers
3	Local / GBP	Only when the client is a local business: live Business Profile data, NAP consistency, reviews (skips itself with a note for national B2B firms)	GBP table or skip note
→	Report + write-back	Assembles the 14-section report, lints it, generates the PDF, then the learning agent stores durable facts for next time	report .md/.html/.pdf + updated memory

4.4 Client memory (why audit #2 is better than audit #1)

The system keeps a **persistent, per-client knowledge store** on disk — independent of any chat session or Claude model:

- **Profile** — what the business is (model, markets, ICPs, seed themes)
- **Facts** — durable, evidence-tagged learnings ("authors are CSS-hidden sitewide — evidence: 4 files cited"), each with confidence + source. Unsourced claims are force-downgraded to low confidence; transient metrics and anything secret-shaped are **rejected**.
- **History** — the audit-score timeline (48 → 57 → ...)
- **Cache** — expensive API responses with freshness windows + provenance, so repeat audits don't re-pay for unchanged data and cached numbers stay citable

Every audit **reads this first** (starts informed) and **writes back after** (gets smarter). See §11 to drive it manually.

5 · Installation

5.1 Three install paths

PATH A — PLUGIN MARKETPLACE NEW (FASTEST; SKILLS + AGENTS)

```
/plugin marketplace add creator-imran/claude-seo-pro
/plugin install claude-seo-pro@creator-imran-claude-seo-pro
```

Requires Claude Code 1.0.33+ and GitHub access to the (private) repo. **Note:** the plugin route installs skills/agents; the Python engines, drift guard and white-label tooling come with the full installer (Path B/C) — use those for a complete operator seat.

PATH B — WINDOWS (POWERSHELL)

```
git clone https://github.com/creator-imran/claude-seo-pro.git
powershell -ExecutionPolicy Bypass -File claude-seo-pro\install.ps1
```

PATH C — MACOS / LINUX

```
git clone https://github.com/creator-imran/claude-seo-pro.git
bash claude-seo-pro/install.sh
```

What the installer does, in order: copies the 30 skills + 19 agents + scripts + the 4 engines into `~/.claude/` → NEW stamps `~/.config/claude-seo/install-manifest.json` with the product version → installs Python report dependencies → offers guided onboarding (skip with `-NoOnboard` / `--no-onboard`).

Prerequisites & what to do if one is missing

REQUIREMENT	CHECK	IF MISSING
Claude Code CLI	<code>claude --version</code>	Install from claude.ai/claude-code
Python 3.10+	<code>python --version</code>	python.org → install → re-open terminal
Git	<code>git --version</code>	<code>winget install Git.Git</code> (Windows) or Xcode tools (macOS)
Node.js (only for DataForSEO/Firecrawl/Exa MCP servers)	<code>node --version</code>	nodejs.org LTS — you can install later; only those MCP servers need it

5.2 Verify the install + the drift guard NEW

From the repo folder, anytime:

```
python tools/check_install.py
```

It hashes every Pro-owned file in the repo against the installed copy and prints one of:

- `[INSTALL FRESH]` installed skills match the repo. — you're current.
- `[INSTALL STALE]` re-run `install.ps1 / install.sh` — with a list of DRIFTED/MISSING files.

WHY THIS EXISTS — A TRUE STORY

A real audit once ran against **stale installed skills** because the installer wasn't re-run after the product gained new features. The audit still "worked" — on the old playbook. The drift guard makes that condition loud instead of silent. **Habit:** run it after every `git pull`.

`/seo-setup verify` also surfaces this check automatically when run from a repo checkout.

5.3 Updating to a new version

```
cd claude-seo-pro
git pull
python tools/check_install.py          # will report STALE
powershell -ExecutionPolicy Bypass -File install.ps1 -NoOnboard  # refresh
python tools/check_install.py          # should now report FRESH
```

Your keys, client knowledge, cache and branding are in `~/.config/claude-seo/` — untouched by updates.

Uninstall

Delete the skill/agent folders the installer created under `~/.claude/skills/` (all `seo*` directories) and `~/.claude/agents/seo-*.md`. To remove your data too (keys, client memory): delete `~/.config/claude-seo/` — this is irreversible; the client knowledge store lives there.

6 · API onboarding — all 7 providers, step by step

Two equivalent ways to run onboarding; both end up in the same secure store:

```
/seo-setup ← in Claude Code (guided, recommended)
python ~/.claude/skills/seo/onboarding/setup_wizard.py ← directly in your terminal
```

SECURITY MODEL IN ONE LINE

Keys are typed into a **masked terminal prompt** (never into chat), validated **live against the provider** before storage, then written to `~/.config/claude-seo/*.json` with owner-only permissions. Nothing is sent anywhere except the provider's own API. `/seo-setup rotate <provider>` replaces a key later.

Status check anytime: `/seo-setup verify` or `python ../setup_wizard.py --check`. Configure a single provider: `--provider dataforseo` etc.

6.1 DataForSEO — the off-site engine (most valuable single key)

Unlocks: rankings, full keyword research (volume/CPC/difficulty/intent), competitor analysis & gap, business listings, AI-mention checks. Without it the audit is on-site only.

- 1 Create an account at `app.dataforseo.com/register`. Top up a small balance (\$10–50 lasts a long time — a full audit's data is typically \$0.10–\$3).
- 2 Your credentials are the **API login (email) + API password** shown on the dashboard — not your site login password.
- 3 Run the wizard; it validates with a \$0 auth call and a ~\$0.01 data probe.

THE CLASSIC TRAP: IP WHITELIST (ERROR 40207)

DataForSEO accounts often have an IP-whitelist enabled. Your key can be **valid** while every *data* call is blocked until your machine's public IP is added at `app.dataforseo.com/api-access`. The validator detects this distinctly and tells you. **On home/mobile connections your IP rotates** — it will break again. Either disable the whitelist, or expect to re-add the new IP (find it: `curl https://api.ipify.org`). Full play-by-play: §17.

What the audit can and can't do at each DataForSEO subscription level

YOU HAVE	WORKS	SHOWS AS "DATA PENDING"
Base account + balance	Rankings, keyword research, competitors, listings, AI mentions	Backlinks (needs the separate Backlinks subscription — error 40204)
+ Backlinks subscription	Everything incl. referring domains / domain rating	—

6.2 Google API key — real Core Web Vitals (free)

Unlocks: PageSpeed Insights + CrUX **field data** — what real Chrome users experience (the ranking signal), not just lab simulations. Free with normal quotas.

- 1 Go to `console.cloud.google.com/apis/credentials` → create/select a project → **Create credentials** → **API key**.
- 2 Enable the **PageSpeed Insights API** for the project (APIs & Services → Library → search "PageSpeed").
- 3 Run the wizard — it validates the key without burning a full Lighthouse run.

6.3 Google Search Console + GA4 (OAuth) — skippable, attach later

Unlocks: indexation status, search performance (clicks/impressions/CTR/position), GA4 organic traffic trends. Needs **owner access** to the client's properties, so the wizard supports **[N]ow / [L]ater / [S]kip** — "Later" marks it pending and audits degrade gracefully ("Data pending — Google OAuth not connected").

Full OAuth setup when you're ready (10 minutes, one time)

- 1 In the same Google Cloud project: enable **Search Console API** and **Google Analytics Data API**.
- 2 OAuth consent screen → External → add yourself as a test user.
- 3 Credentials → Create → **OAuth client ID** → **Desktop app** → download `client_secret.json`.
- 4 Run the wizard with the file path; a browser opens for consent; the token is stored owner-only. The `client_secret` itself is read from your file at runtime, never copied into the token.

6.4 Google Business Profile (OAuth) — for local-business clients

Unlocks: first-party local insights (impressions, calls, direction requests). Needs the profile owner's consent. **If skipped**, local audits automatically fall back to DataForSEO's public listing data — labeled accordingly. Same OAuth pattern as §6.3; additionally requires GBP API access approval from Google (the wizard explains).

6.5 Firecrawl — large/JS-heavy site crawling

Unlocks: full-site crawling with JavaScript rendering — for SPAs and very large sites where the built-in fetcher isn't enough. Key from `firecrawl.dev/app/api-keys`; the validator checks your credit balance.

6.6 Exa — neural search for competitor/entity research

Unlocks: semantic web search used in competitor discovery and entity research. Key from `dashboard.exa.ai/api-keys`; validated with a single 1-result probe.

6.7 Slack — the chat connector

Unlocks: running audits from Slack (`/seo audit client.com` in a channel). Needs a Slack app's **signing secret** + **bot token**; the validator calls `auth.test`. Full deployment walkthrough in §13 — configure the provider here first, deploy the bridge there.

6.8 OpenRouter — the backend fallback NEW 1.2.0

Unlocks: a second model backend. When Claude credits run out, switch the whole system to OpenRouter in ~60 seconds and keep working — or run other frontier models by choice. **Fully optional and deferrable:** the wizard offers **[N]ow / [L]ater / [S]kip**; nothing else depends on it.

- 1 Create a key at `openrouter.ai/keys` (starts with `sk-or-`) and add some credit.
- 2 Run the wizard (`/seo-setup` , or directly: `python ~/.claude/skills/seo/onboarding/setup_wizard.py -provider openrouter`). Validation uses OpenRouter's free key-info endpoint and **shows your remaining credit**.
- 3 That's it — the key is stored owner-only. Actually *switching* backends is a separate, deliberate step: see §12.5.

MINIMUM VIABLE SETUP

Just **DataForSEO + Google API key** gives you the full 4-phase audit with live rankings, keyword research and real CWV. Everything else adds depth and can be attached later without re-doing anything.

7 · Your first full audit — a complete walkthrough

Type this in Claude Code and press `Enter`:

```
/seo audit https://yourclient.com
```

Here is what happens, phase by phase, with what you should expect to **see** — so you can tell healthy progress from a problem:

Phase 0 — Business intelligence (1–2 min)

Claude first recalls the client's knowledge store ("*What we know about X... 12 learned facts... last score 57*" — or "no prior knowledge" on a first run), then infers the business profile from the homepage and confirms anything uncertain with you. **You may be asked 1–2 questions** ("Is KSA a real target market or just a translated page?") — answering well here makes Phase 2's keyword research dramatically better targeted.

Phase 1 — Evidence capture + parallel specialists (5–10 min)

You'll see pages being fetched to disk (dozens of URLs, each with an HTTP status), then several specialist agents launching **in parallel**. Expect names like `seo-technical`, `seo-content`, `seo-schema`, `seo-geo` with running token counts. Cheap models handle extraction; stronger ones do reasoning — this is the model router (\$12) saving you money automatically.

HEALTHY SIGNS

"50 fetches, 50 OK" · agents citing file paths · "adversarially verifying finding..."

WARNING SIGNS

fetch failures piling up (site blocking — the audit will say so honestly) · an agent reporting it cannot read its evidence (see §17).

Phase 2 — Live keyword research (2–5 min, ~\$0.05–\$1)

DataForSEO calls fan out: your rankings, position distribution, competitor set, keyword demand, the gap vs a real competitor. Every response is saved to disk — those files are the citations behind the report's tables.

Phase 3 — Local/GBP (skips itself for non-local businesses)

National B2B firm? You'll see a one-line skip note (that's contract-correct, not an omission). Local business? Live GBP data: rating, reviews, categories, photos, NAP comparison.

Assembly → linter → PDF → memory write-back (2–4 min)

The report is assembled to the 14-section contract, the linter checks it (\$8.2), the PDF is generated, and the learning agent proposes durable facts to remember (preview-then-ingest). The run ends by stating **where every deliverable is**.

DELIVERABLE	WHAT IT'S FOR
FULL-AUDIT-REPORT.md	Working markdown — grep-able, diff-able
ACTION-PLAN.md	Prioritized actions with effort estimates
<client>-seo-audit-<YYYY-MM>.pdf	The client deliverable — white-labeled, contract-complete
Evidence folder + saved API responses	The receipts — keep them with the engagement
Knowledge-store update	Audit #2 starts informed

Cost anatomy of a real audit (receipts from production)

A real full live-data audit (Dubai client, 5 locales): **28 DataForSEO calls, \$0.61 total**. A US B2B audit's data: ~\$0.10. Claude token costs depend on site size; the router keeps extraction on the cheapest tier. Preview keyword-research cost before spending: `python ~/.claude/skills/seo/scripts/keyword_research.py --plan --seeds "..." --markets "US:en"`.

8 · Reading & delivering the report

8.1 The 14 sections, and how to read each one

#	SECTION	HOW TO READ IT / WHAT TO CHECK
1	Cover	Client, scope, version, date. White-labeled with <i>your</i> brand (\$8.4).
2	Data Integrity & Methodology	Read this first with a skeptical client. How evidence was gathered; every pending data source with its exact error + unlock step.
3	Executive Summary	The story in one page: 4 KPI cards, the score ring, the full category scorecard (PENDING rows greyed — honesty, not failure), the 5 highest-leverage moves.
4	Priority Issues	Finding cards: severity → explanation → verbatim evidence block → fix → source. If a finding has no evidence line, it shouldn't be there — that's the contract.
5	What's Genuinely Strong	Mandatory. A findings-only report reads as a hit job and loses the client.
6	Organic Search Visibility	Position-distribution buckets (each with a "read" column) + the ≥10-row quick-wins table. Watch for the honesty pattern: quick-wins labeled "brand/noise" when that's the truth.
7	Keyword Research	Seeds-by-market volumes/CPC, the competitor-gap table, opportunity tiers. Thin secondary markets are <i>said</i> to be thin.
8	Local SEO & GBP	Field-by-field GBP table + cross-source NAP check — or an explicit one-line skip note for non-local clients.
9	Competitive Landscape	Who shares your keyword footprint, <i>classified</i> (direct competitor vs directory/platform — you don't out-rank Wikipedia, you learn from it).
10	On-Page Detail	One row per audited money page — all of them. Title/desc lengths, H1s, per-page issues, cannibalization patterns, and what's confirmed clean.
11	Performance (field vs lab)	Two separate tables. Field = the ranking signal; lab = diagnostic only. Lab artifacts are labeled (an 18s lab LCP next to a 2.4s field LCP is throttling, not reality).
12	Per-category detail	Content/E-E-A-T, Schema, AI-readiness each get full depth — this is where the specialists' work lands.
13	Action Plan	Quick-wins + strategic tables (effort + data-backed why) + sequencing + the "data to unlock" list.
14	Appendix	The receipts: every API call with status, integrity controls, evidence corpus, corrections vs any prior report.

8.2 The contract linter NEW

Before any report ships, lint it (the audit does this automatically; you can run it yourself on any report):

```
python tools/lint_report.py path\to\report.html          # human verdict
python tools/lint_report.py path\to\report.html --json  # machine-readable
```

VERDICT	MEANING	YOUR MOVE
PASS	All 14 sections present, no leftover placeholders, depth floors met	Deliver
WARN	Structure OK; depth floors or pending-cause notes flagged	Review the warnings, then deliver
FAIL	Missing sections / unfilled placeholders / summary-only compression	Do not deliver. Fix, re-lint

8.3 PDF generation

Automatic at the end of an audit. Manually, from any report HTML (Windows):

```
msedge --headless --disable-gpu --no-pdf-header-footer --print-to-pdf="out.pdf"
"file:///C:/path/report.html"
```

macOS/Linux: the equivalent `chrome --headless`. (WeasyPrint via `scripts/google_report.py` is an alternative chart-heavy generator when its dependencies are installed.)

8.4 White-labeling NEW — your brand on every report

```
python ~/.claude/skills/seo/onboarding/branding.py show
python ~/.claude/skills/seo/onboarding/branding.py set --preparer "Acme Digital" --primary
"#16324f" --accent "#d4740e" --footer "Acme Digital – confidential" --logo "C:/acme/logo.png"
python ~/.claude/skills/seo/onboarding/branding.py reset
```

Settings persist in `~/.config/claude-seo/branding.json`; every subsequent report picks them up (cover, colors, preparer line, footer, optional logo). Unset = clean neutral product branding. Invalid colors fall back safely — a typo can't break a client deliverable.

9 · Pro command reference

`/seo-setup` — onboarding · verify · rotate

```
/seo-setup          # guided onboarding, provider by provider
/seo-setup verify    # per-provider status + install-freshness check
/seo-setup rotate dataforseo # replace one key safely
```

`/seo-knowledge` — the client memory

```
/seo-knowledge recall yourclient.com # the briefing: profile + facts + score history
/seo-knowledge remember yourclient.com # record a fact (you'll be asked for evidence!)
/seo-knowledge cache                  # cache stats / purge
```

EXPECT THIS

Facts you add **without evidence** are stored at forced-low confidence; transient metrics ("score is 61 right now") and anything that looks like a secret are **rejected**. That's the hygiene guard, not a bug.

`/seo-learn` — manual learning pass

```
/seo-learn yourclient.com
```

Distills durable facts from the latest audit evidence — preview first, then ingest. Runs automatically after full audits; run manually after big client conversations or site changes.

`/seo-models` — model routing & cost

```
/seo-models          # show the routing policy + per-tier costs
```

Details and overrides in §12.

`/seo-connect` — the Slack connector

Configuration status, command surface, deployment guidance. Full deployment in §13.

10 · Upstream command reference (27 commands)

UPSTREAM — inherited intact; the Pro layer feeds them better data and memory. Grouped by job:

AUDIT & ANALYSIS

COMMAND	USE WHEN...
<code>/seo audit <url></code>	The full 4-phase audit (\$7). The flagship.
<code>/seo page <url></code>	One page, deeply — before/after a rewrite, landing-page checks
<code>/seo technical <url></code>	Crawlability/indexability/security/CWV across 9 categories, alone
<code>/seo content <url></code>	E-E-A-T + content quality + AI-citation readiness, alone
<code>/seo schema <url></code>	Detect → validate → generate Schema.org JSON-LD
<code>/seo images <url></code>	Alt text, formats, lazy-loading, CLS prevention, image SERP
<code>/seo sitemap <url generate></code>	Audit an XML sitemap or generate one from templates
<code>/seo sxo <url></code>	"Why doesn't this well-optimized page rank?" — SERP-backwards intent analysis

AI SEARCH (GEO)

<code>/seo geo <url></code>	AI Overviews / ChatGPT / Perplexity readiness: crawler access, citability scoring, entity clarity — evidence-based (it will tell you llms.txt is <i>not</i> a confirmed citation lever)
-----------------------------------	---

LOCAL

<code>/seo local <url></code>	GBP, citations, NAP, reviews, local schema, map pack
<code>/seo maps [command]</code>	Geo-grid rank tracking, GBP audit via API, review intelligence, competitor radius

RESEARCH & STRATEGY

<code>/seo plan <type></code>	Strategic plan by industry (saas, local, ecommerce, publisher, agency)
<code>/seo cluster <keyword></code>	SERP-overlap topic clustering → hub-and-spoke architecture
<code>/seo content-brief <topic></code>	Competitive content briefs with per-section word counts
<code>/seo competitor-pages <url></code>	"X vs Y" / "alternatives to X" page generation
<code>/seo programmatic <url></code>	Pages-at-scale planning with thin-content safeguards
<code>/seo backlinks <url></code>	Link profile via Moz/Bing/CommonCrawl (free) or DataForSEO
<code>/seo hreflang <url></code>	International/i18n audit + correct hreflang generation
<code>/seo ecommerce <url></code>	Product schema, Shopping/marketplace visibility
<code>/seo flow [stage]</code>	Evidence-led FLOW framework prompts (41, CC BY 4.0)

DATA & MONITORING

<code>/seo google [command]</code>	GSC / PageSpeed / CrUX history / Indexing / GA4 directly
<code>/seo dataforseo [command]</code>	Live SERP/keyword/competitor data ad hoc
<code>/seo drift baseline compare history <url></code>	Snapshot SEO-critical elements; detect regressions later (\$14)
<code>/seo firecrawl [command] <url></code>	Full-site crawl / site map (extension)
<code>/seo image-gen [use-case]</code>	OG images, blog heroes via Gemini (extension)
<code>/seo bing</code> , <code>/seo unlighthouse</code> , <code>/seo ahrefs</code> , <code>/seo seranking</code> , <code>/seo profound</code>	Further extensions: Bing/IndexNow, local multi-page Lighthouse, Ahrefs data, AI share-of-voice, LLM citation tracking

11 · Client knowledge & learning — the system that gets smarter

Conversational use is \$9 (`/seo-knowledge` , `/seo-learn`). Under the hood, per client (slug of the domain):

```
~/ .config/claude-seo/clients/yourclient-com/  
  profile.json    ← business understanding (model, markets, ICPs, seeds)  
  facts.jsonl     ← evidence-tagged durable facts (append-only, dedup, supersede)  
  history.jsonl   ← audit-score timeline  
~/ .config/claude-seo/cache/<provider>/ ← cached API responses + provenance
```

WHAT MAKES A FACT "DURABLE" (THE GUARDS)

CANDIDATE	OUTCOME	WHY
"Targets KSA as a secondary market" + evidence	✅ stored (high)	Durable + sourced
"Targets enterprise clients" (no evidence)	⚠️ stored at low	Unsourced → forced low confidence
"Performance score is 30 right now"	❌ rejected	Transient metric → belongs in history, not facts
"API key is Alza..."	❌ rejected	Secret-shaped content never enters memory

When the truth changes, facts are **superseded**, not duplicated — memory gets more accurate, not just bigger. The cache refuses stale reads (expired = miss = re-fetch) and keeps provenance so even cached numbers stay citable.

Power use: the CLI underneath

```
python ~/.claude/skills/seo/knowledge/store.py recall yourclient.com  
python ~/.claude/skills/seo/knowledge/store.py add-history yourclient.com --score 61 --note  
"post-CLS-fix re-audit"  
python ~/.claude/skills/seo/knowledge/learn.py preview --domain yourclient.com --file  
candidates.json  
python ~/.claude/skills/seo/knowledge/cache.py stats
```

BACK THIS UP

`~/ .config/claude-seo/clients/` is your accumulated client intelligence. Include it in your backups (see \$16). It is deliberately NOT in the repo.

12 · Model routing & cost control

Each audit task runs on the cheapest Claude model that does it well, decided at dispatch time. The main conversation stays on your session's model (switching it would invalidate the prompt cache — slower and pricier).

TIER	MODEL	AUDIT WORK
extraction	Haiku 4.5	Titles/metas/H1s/status/schema-presence from pre-fetched files
verification	Sonnet 4.6	Adversarial double-check of Critical/High findings
reasoning	Sonnet 4.6	E-E-A-T judgment, intent analysis, fact extraction
synthesis	Opus 4.8	Cross-dimension prioritization, the report itself
orchestration	Opus 4.8 (pinned)	The main loop

Real example: a 60k-token extraction task costs **\$0.11 on Haiku vs \$0.55 on Opus** — multiplied across every specialist, every audit. Inspect and override:

```
/seo-models                                # show policy
python ~/.claude/skills/seo/routing/model_router.py route --agent seo-technical
python ~/.claude/skills/seo/routing/model_router.py estimate --tier extraction --in 60000 --out 10000
python ~/.claude/skills/seo/routing/model_router.py set --force-model opus      # flagship-quality pass
python ~/.claude/skills/seo/routing/model_router.py reset
```

WHY CHEAP MODELS ARE SAFE HERE

Extraction agents read **pre-fetched files** — there's nothing to hallucinate about; they're transcribing. Judgment work is routed up-tier. One observed limit: very large minified pages (300KB+) can exhaust a small model's turn budget — if a specialist stalls, that's the first suspect (\$17).

12.5 Switching backends: Claude credits ↔ OpenRouter NEW 1.2.0

The system always runs on Claude Code; what switches is the **backend** the CLI talks to. One tool manages it — installed at `~/.claude/skills/seo/scripts/switch_provider.py` (same file as `tools/switch_provider.py` in a repo checkout). Conversationally: `/seo-provider`.

RECIPE 1 — "I RAN OUT OF CLAUDE CREDITS" (~60 SECONDS)

```
python ~/.claude/skills/seo/scripts/switch_provider.py use openrouter
```

- 1 In Claude Code: `/logout` (clears cached Anthropic auth — skipping this causes auth conflicts).
- 2 **Exit Claude Code completely**, then start it again: `claude`. The endpoint is read **once at startup** — without the restart, nothing changes.

- 3 Keep working. The default profile maps every tier to the **same Claude models, billed via OpenRouter** — near-zero quality risk, slight cost premium vs first-party (this is continuity, not savings).

RECIPE 2 — SWITCHING BACK TO CLAUDE CREDITS

```
python ~/.claude/skills/seo/scripts/switch_provider.py use anthropic
```

Then: restart Claude Code → `/login` . Done.

RECIPE 3 — CUSTOM MODELS (FRONTIER-GRADE ONLY)

```
python ~/.claude/skills/seo/scripts/switch_provider.py set-models --opus moonshotai/kimi-k2.6 -  
-sonnet moonshotai/kimi-k2.6 --haiku <fast-model-slug>  
python ~/.claude/skills/seo/scripts/switch_provider.py use openrouter --profile custom
```

Pick slugs from `openrouter.ai/models` — every slug is **validated live** before anything is written (typos get close-match suggestions). Then the same `/logout` → restart steps.

THE QUALITY CONTRACT — READ ONCE

Claude Code and this product's Evidence Integrity Protocol are **optimized for Anthropic models**; on other models, adherence to the evidence discipline is not guaranteed. Use **frontier-grade models only** (Kimi K2.6 class). Backstops that hold regardless: the report linter still enforces structure, and every report's Data Integrity section **discloses which backend produced it**.

OPERATIONS

```
python ~/.claude/skills/seo/scripts/switch_provider.py status    # backend, model map, key  
health, remaining credit  
python ~/.claude/skills/seo/scripts/switch_provider.py restore  # undo: restore settings.json  
from latest backup
```

Safety, built in: the key AND the model map are validated live **before** writing (a dead key can't brick your next session); `~/.claude/settings.json` is backed up on every write (last 5 kept); exactly 7 env keys are managed and nothing else is touched; a foreign gateway URL is never overwritten without `--force` .

13 · Slack connector — audits from chat

For the manager who doesn't live in a terminal: type `/seo audit client.com` in Slack; the bridge verifies, authorizes, acknowledges in under 3 seconds, runs a headless audit, and posts the result back.

SECURITY MODEL (READ BEFORE DEPLOYING)

- **Signature verification** — every request HMAC-checked against your signing secret with a 5-minute replay window; forged/stale requests get 401.
- **Deny-by-default authorization** — nobody can trigger audits until you allow-list their user/channel ID. Audits cost money; this is intentional.
- **Secrets stay host-side** — the bot token/signing secret live in `~/.config/claude-seo/slack.json` on the machine running the bridge, never in Slack.
- **Fixed command surface** — only commands you enable parse at all; URL targets are validated.

Deployment walkthrough (one-time, ~30 min)

- 1 Create a Slack app (api.slack.com/apps) → add a slash command `/seo` pointing at your bridge URL → install to workspace. Note the **signing secret** and **bot token**.
- 2 Onboard the slack provider (\$6.7) so the secrets are stored.
- 3 Configure operating policy in `~/.config/claude-seo/connector.json`: `enabled_commands` (e.g. `["audit", "page"]`), `allowed_users` (Slack user IDs), model, timeout.
- 4 Run the bridge on a host that has Claude Code installed and is reachable by Slack (or via a tunnel):

```
python ~/.claude/skills/seo/connector/slack_bridge.py --port 8743
python ~/.claude/skills/seo/connector/slack_bridge.py --selftest # offline sanity first
```

- 5 In Slack: `/seo audit https://client.com` → expect the "running..." ack, then the result post. Unauthorized users get a polite denial (by design).

Full deploy doc: `docs/CONNECTOR.md`. WhatsApp: planned as an adapter on the same runner.

HONEST STATUS

The connector's security core (signatures, authorization, parsing, runner) is unit-tested in CI. The live Slack round-trip is **your deployment step** — it needs your Slack app and a reachable host.

14 · Drift monitoring — "did anything break?"

Git for your client's on-page SEO. Capture a baseline after every audit/fix cycle; compare any time something ships:

```
/seo drift baseline https://client.com      # snapshot SEO-critical elements (SQLite)
/seo drift compare https://client.com      # diff current site vs baseline, severity-tagged
/seo drift history https://client.com      # the change timeline
```

The workflow that earns retainers: baseline after delivery → client's dev team ships things → `compare` before your monthly check-in → walk in saying "your release on the 14th dropped canonical tags on 12 pages" with the diff to prove it. Pair with the knowledge store's score history for the long-run trend conversation.

15 · Staying current — sync, CI, versions

UPSTREAM SYNC (YOU BENEFIT FROM THE OPEN-SOURCE PROJECT FOREVER)

```
python tools/sync_upstream.py --dry-run      # preview upstream changes (also the provenance check)
python tools/sync_upstream.py --tag vX.Y.Z   # vendor a new upstream release
```

Pro-owned files are protected by a skip-list; the one in-place patch re-applies as an idempotent overlay that **fails loudly** if upstream moves an anchor. A weekly GitHub Action does this automatically and opens a reviewable PR — never auto-merges. A clean dry-run reports **exactly one** changed file (the managed overlay): that's your proof the fork is still *upstream + overlay + additions*.

CI NEW

Every push/PR runs: the 68-assertion adversarial component suite · repo integrity (compile-all, JSON validity, secret scan, no client data) · overlay check · report-linter self-test. All stdlib, offline, no keys needed. Check the repo's **Actions** tab — green before you pull, green before you deploy to a client seat.

VERSIONING

`VERSION.md` is the human ledger (what's upstream vs Pro, version history); `system-version.json` is the machine manifest; the installed stamp is in `~/.config/claude-seo/install-manifest.json`. The roadmap — including the **refused-features list** (no hosted dashboard, no own crawler, no multi-tenant SaaS: they'd destroy the local-first differentiation) — is `docs/ROADMAP-7-to-9.md`.

16 · Maintenance & operations

CADENCE	TASK	HOW
After every <code>git pull</code>	Refresh + verify install	<code>install.ps1 -NoOnboard</code> → <code>python tools/check_install.py</code> → FRESH
Monthly	Re-pull rankings; drift compare per client	<code>/seo dataforseo</code> · <code>/seo drift compare</code>
~28 days after CWV fixes	Re-measure field data	CrUX is a trailing window — earlier re-tests mislead
Quarterly	Rotate API keys	<code>/seo-setup rotate <provider></code>
Weekly (automated)	Upstream-sync PR review	GitHub Actions opens it; you review & merge
Always	Back up client intelligence	Include <code>~/.config/claude-seo/</code> in backups (keys + knowledge + branding)

Moving to a new machine / setting up a second seat

1. Clone the repo, run the installer.
2. Run `/seo-setup` — keys are re-entered per machine (they are deliberately not in the repo; each seat holds its own).
3. To carry client memory: copy `~/.config/claude-seo/clients/` from the old machine.
4. `python tools/check_install.py` → FRESH; `/seo-setup verify` → green.

17 · Troubleshooting encyclopedia

Errors you may actually meet, with cause and fix. Every one of these has been hit (and solved) in real operation.

SYMPTOM / CODE	CAUSE	FIX
40207 Access denied. Your IP is not whitelisted — DataForSEO auth OK but every data call fails	The account's IP-whitelist doesn't include this machine's current public IP. Dynamic IPs rotate — this recurs after router restarts/mobile hotspots.	Get your IP (<code>curl https://api.ipify.org</code>) → add at <code>app.dataforseo.com/api-access</code> → retry. Recurs often? Disable the whitelist. The validator and the audit both detect this distinctly and will not fabricate around it.
40204 ... Backlinks subscription required	DataForSEO Backlinks is a separate subscription	Activate it (<code>app.dataforseo.com/backlinks-subscription</code>) or supply Moz/Bing creds — until then the Backlinks section honestly shows Data pending
40501 Invalid Field on Labs calls for some country/language pairs	DataForSEO Labs doesn't support every locale combination (e.g. en-language ideas in KSA/Qatar)	The keyword module falls back to Google Ads volume data for those markets and says so in the report's method notes
Audit report shows "Data pending — Google OAuth not connected"	GSC/GA4 onboarding was skipped/deferred	Run <code>/seo-setup</code> , choose the Google OAuth provider, complete the browser consent (\$6.3)
PSI / CWV check times out	PageSpeed runs a live Lighthouse — 60s is often not enough	Re-run; allow ~2 minutes per strategy (mobile/desktop). One strategy at a time helps
Lab CWV terrifying (LCP 18s) but field fine	Lab throttling artifact, not reality	Trust field (CrUX) — it's the ranking signal. The report labels lab as diagnostic-only by contract
[INSTALL STALE] from check_install / odd audit behavior after an update	Repo updated but installer not re-run — audits run on old skills	<code>install.ps1 -NoOnboard</code> (or <code>install.sh</code>) → re-check → FRESH. This exact incident happened in production; the guard exists because of it
Report linter FAIL: "unfilled template placeholders"	A generated report still contains <code>{{TOKENS}}</code>	The generation step missed substitutions — re-run report assembly; never hand-deliver a FAILED report
Report linter FAIL: missing sections	Summary-only report (the pre-contract failure mode)	Re-assemble against <code>references/report-template.md</code> ; every section is mandatory (pending ≠ omitted)
A specialist agent stalls on a huge page	300KB+ minified HTML can exhaust a small model's turn budget	Re-run that dimension; if persistent, force a stronger tier: <code>/seo-models</code> → force-model sonnet for the run
Slack: 401 invalid signature	Wrong signing secret, or request older than the 5-min replay window	Re-check the secret in <code>slack.json</code> matches the app; check host clock skew
Slack: "you're not authorized to run audits"	Deny-by-default — user/channel not on the allow-list	Add the Slack user ID to <code>connector.json</code> → <code>allowed_users</code> (intentional friction; audits cost money)

OAuth browser window never opens (headless/server)	OAuth needs a human browser consent	Run onboarding once on a desktop machine; copy the token from <code>~/.config/claude-seo/</code> to the server seat
PowerShell: piping output into <code>python -c</code> gives JSON errors	Known PS quirk — stdin arrives empty	Write to a file and read it, or use <code>Select-String</code> on the output instead of piping into Python
<code>git push</code> rejected / "not a repository"	Folder not initialized or remote missing	See <code>docs/PUBLISH.md</code> ; the publish script handles init + remote + secret-guard in one step
Switched to OpenRouter but nothing changed / still billing Claude	The endpoint env is read once at Claude Code startup — the session wasn't restarted	Exit Claude Code completely and start it again (Recipe 1, §12.5). <code>status</code> shows what settings.json actually says
Auth errors right after switching backends	Cached Anthropic login conflicting with the OpenRouter token (or vice versa)	<code>/logout</code> before switching to OpenRouter; <code>/login</code> after switching back to Anthropic
Switcher refuses: "unknown model slug"	Typo or model no longer listed on OpenRouter	Use the close-match suggestions it prints, or pick from <code>openrouter.ai/models</code> ; slugs are validated live by design
Session broken after a backend switch	Rare — e.g. settings edited by hand alongside the switcher	<code>python</code> <code>~/.claude/skills/seo/scripts/switch_provider.py</code> <code>restore</code> (latest backup), restart Claude Code

WHEN SOMETHING ELSE BREAKS

Ask Claude in-session — it has the full system context. Include the exact error text. For reproducible bugs, the CI suite (`python tests/test_owned_components.py`) tells you in seconds whether the core is healthy on your machine.

18 · Security & privacy model

ASSET	WHERE IT LIVES	PROTECTION
API keys	<code>~/.config/claude-seo/*.json</code>	Owner-only (0600/ACL) · entered masked in terminal · never in chat, repo, or git · validated then stored
OAuth tokens	same store	client_secret read from your file at runtime, never copied into tokens
Client knowledge & cache	<code>~/.config/claude-seo/clients cache/</code>	Owner-only · gitignored belt-and-suspenders · secret-shaped content rejected by the learn guards
Slack secrets	<code>slack.json</code> host-side	HMAC verification + replay window + deny-by-default authorization
The repo	GitHub (private)	CI secret-scans every push; publish script has a staged-secret guard; tracked-file scan is clean
Network	—	Calls go only to each provider's own endpoint. No telemetry, no third-party relay. SSRF/DNS-rebinding-safe fetchers inherited from upstream (83 bypass-class tests)

Threat model and removal procedures: `docs/SECURITY.md` .

19 · FAQ

Do I need to know how to code?

No. You type `/seo` commands in Claude Code and occasionally paste a command into a terminal (this manual gives you every one verbatim). The system was built for exactly this profile.

Does it work with zero API keys?

Yes — the on-site audit (technical, content, schema, on-page, AI-readiness) runs from captured HTML alone. Keys add the off-site picture; each missing source becomes an honest "Data pending" section, never a guess.

What does an audit cost?

API data: typically \$0.10–\$3 (real receipt: 28 calls, \$0.61, five locales). Claude tokens: depends on site size; the model router keeps the bulk on the cheapest tier. Preview keyword costs before spending with `--plan` .

Why does the report say "Data pending" instead of just showing a number?

Because the alternative is making the number up. Pending sections carry the exact error and the unlock step — fix the access, re-run, the section fills in. This is the product's core promise.

Can I rebrand reports for my agency?

Yes — \$8.4. One command sets preparer/colors/logo/footer; every later report uses it. No per-report work, no template editing.

How do upstream improvements reach me?

A weekly Action vendors the latest upstream release, protects all Pro files, re-applies the managed patch, and opens a PR for your review. Manually: `python tools/sync_upstream.py --tag vX.Y.Z`.

Multiple clients — do they see each other's data?

No. Knowledge is stored per client (per-domain folders). Reports are generated per client. Nothing is multi-tenant because nothing is hosted — it's all on your machine.

Is my client's data sent anywhere?

Page content goes to Anthropic (that's Claude analyzing it) and queries go to the data providers you enabled (DataForSEO, Google). Nothing else. No analytics, no telemetry, no vendor cloud holding your audits.

What's deliberately NOT in this product?

A hosted dashboard, multi-tenant SaaS, our own crawler, SLAs — see the refused-features list in `docs/ROADMAP-7-to-9.md`. They'd destroy the local-first differentiation that makes this trustworthy.

20 · Glossary

CWV / field vs lab	Core Web Vitals. <i>Field</i> = real Chrome users (CrUX, the ranking signal); <i>lab</i> = simulated throttled load (diagnostic only)
CLS / LCP / INP / TTFB	Layout shift (≤ 0.1 good) / largest paint ($\leq 2.5s$) / interaction delay ($\leq 200ms$) / server response time
E-E-A-T	Experience, Expertise, Authoritativeness, Trust — Google's content-quality lens
GEO / AI search	Optimizing to be cited by AI Overviews/ChatGPT/Perplexity — per Google, the same ranking systems as classic search
KD / Vol / CPC / intent	Keyword difficulty (0–100) / monthly searches / ad cost-per-click / what the searcher wants (informational, commercial...)
GBP / NAP	Google Business Profile / Name-Address-Phone consistency across the web
Schema / JSON-LD	Structured data describing your entity/content to machines
Overlay	The one managed patch this distribution applies to an upstream file, re-applied automatically on sync
Evidence Integrity Protocol	The 8 anti-fabrication rules (§4.2)
Knowledge store	The persistent per-client memory (§11)
Drift (install) / drift (SEO)	Installed skills $\neq$ repo (§5.2) / the site changed vs baseline (§14)
MCP	Model Context Protocol — how live-data servers (DataForSEO, Firecrawl, Exa) plug into Claude Code

21 · Appendix

FILE MAP (OPERATOR'S VIEW)

```
claude-seo-pro/           ← the repo (the product; no client data ever)
  skills/ agents/ scripts/ ← the SEO system (30 skills / 19 agents / 51 scripts)
  onboarding/ knowledge/ routing/ connector/ ← the 4 Pro engines
  tools/                  ← check_install · lint_report · sync_upstream · apply_overlay
  tests/ + .github/       ← CI gate
  docs/ manual/ VERSION.md ← documentation + provenance ledger
~/ .claude/               ← installed copy (refresh with the installer)
~/ .config/claude-seo/    ← YOUR keys + client knowledge + cache + branding (back this up)
```

MANUAL VERSION HISTORY

MANUAL	PRODUCT	NOTES
v02 (this)	1.1.0	Complete rewrite: interactive HTML (sidebar, filter, collapsibles), per-command reference, full onboarding walkthroughs, troubleshooting encyclopedia, white-label & linter & drift-guard coverage
v01	1.0.0	Initial manual — superseded; retained in git history

CREDITS

Built on the MIT-licensed **Claude SEO** by Agrici Daniel (see NOTICE and CONTRIBUTORS.md). Pro distribution by creator-imran. This manual: MIT, same as the product.